

Phishing Detection System

AI-Powered Browser Extension for Real-Time Phishing Detection

A ten-week phased plan covering dataset collection, feature engineering, ML model training, explainable AI, backend services, and a browser extension for real-time phishing detection — plus a threat-intelligence layer, a user-feedback loop, and an admin dashboard.

Roadmap at a Glance

Week	Phase
1	Research & Dataset
2	Feature Engineering
3	Train the ML Model
4	Explainable AI
5	Backend
6	Browser Extension
7	HTML Analyzer
8	Threat Intelligence
9	User Feedback
10	Dashboard

PHASE 1

Research & Dataset

Week 1

Goal

Build the dataset and train the first ML model.

Topics to Learn

- How phishing works

- Common phishing techniques
- URL features
- HTML/DOM features
- Browser extensions (Chrome Manifest V3)

Datasets to Collect

- PhishTank (phishing URLs)
- OpenPhish
- Tranco Top 1M (legitimate sites)

Target

- 50,000 phishing URLs
- 50,000 legitimate URLs

PHASE 2

Feature Engineering

Week 2

Don't feed raw URLs to the model — convert every website into numerical features instead. For example, this URL:

`https://paypal-login-security.xyz/login.php`

Feature	Value
URL Length	41
Number of Dots	2
HTTPS	1
Contains "login"	1
Contains IP Address	0
Hyphens	3
Digits	0
TLD Risk (.xyz)	1
Entropy	5.9

Note: Repeat this feature-extraction process for every URL in the dataset.

Additional Features to Add

- Domain age
- SSL validity
- Number of forms
- Hidden iframes
- Password fields
- JavaScript redirects
- External scripts
- Number of images
- External CSS, etc.

Note: Aim for around 100 features in total.

PHASE 3

Train the ML Model

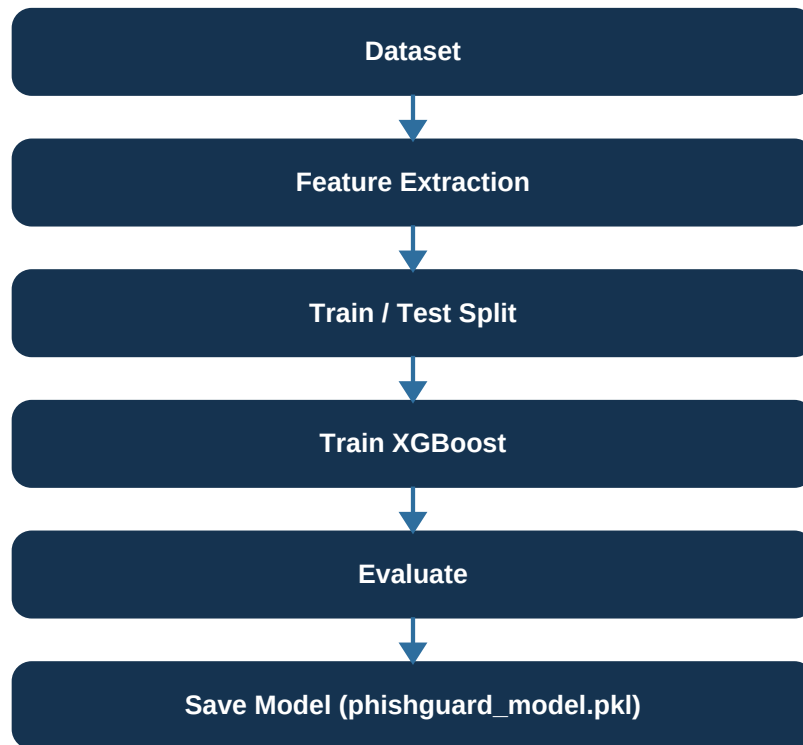
Week 3

Use Python.

Libraries

- pandas
- numpy
- scikit-learn
- xgboost
- lightgbm
- joblib

Pipeline



Target Metrics

- Accuracy > 97%
- Precision > 97%
- Recall > 97%
- F1 Score > 97%

Note: Don't rely only on accuracy — phishing datasets can be imbalanced.

PHASE 4

Explainable AI

Week 4

Instead of just predicting “Phishing,” generate explanations using SHAP.

Example Output

Reason	Contribution
URL contains suspicious keyword	+28%
Domain age only 3 days	+20%
Hidden iframe	+15%

Note: This becomes one of the strongest features of the project.

PHASE 5

Backend

Week 5

Build a REST API.

Suggested Structure

```
server/  
  - controllers/  
  - routes/  
  - services/  
  - ml/  
  - database/  
  - models/  
  - middleware/
```

Endpoints

```
POST    /predict  
POST    /feedback  
GET     /history  
GET     /stats
```

Backend Responsibilities

- Loads the trained model
- Accepts extracted features
- Predicts phishing probability
- Returns explanations
- Stores user feedback

PHASE 6

Browser Extension

Week 6

Suggested Structure

```
extension/  
  - manifest.json  
  - background.js  
  - content.js  
  - popup.html
```

- popup.js
- styles.css
- icons/

Responsibilities

- Detect page changes
- Extract URL
- Extract HTML/DOM features
- Send features to backend
- Display risk score
- Show explanations
- Collect user feedback

PHASE 7

HTML Analyzer

Week 7

This is where the project becomes more interesting: the extension inspects the DOM and extracts features such as:

- Password fields
- Login forms
- External scripts
- Hidden iframes
- Redirect scripts
- Clipboard API usage
- Notification permission requests
- Fullscreen requests

Note: These become additional ML inputs.

PHASE 8

Threat Intelligence

Week 8

If the ML score is uncertain (for example, between 40% and 70%), query external services such as:

- Google Safe Browsing
- VirusTotal

- PhishTank

Note: This saves API usage, since not every website needs to be queried.

PHASE 9

User Feedback

Week 9

After each prediction, let users report:

- “This is phishing”
- “This is safe”

Store

- URL
- Prediction
- User feedback
- Timestamp

Note: Later, retrain the model with this data.

PHASE 10

Dashboard

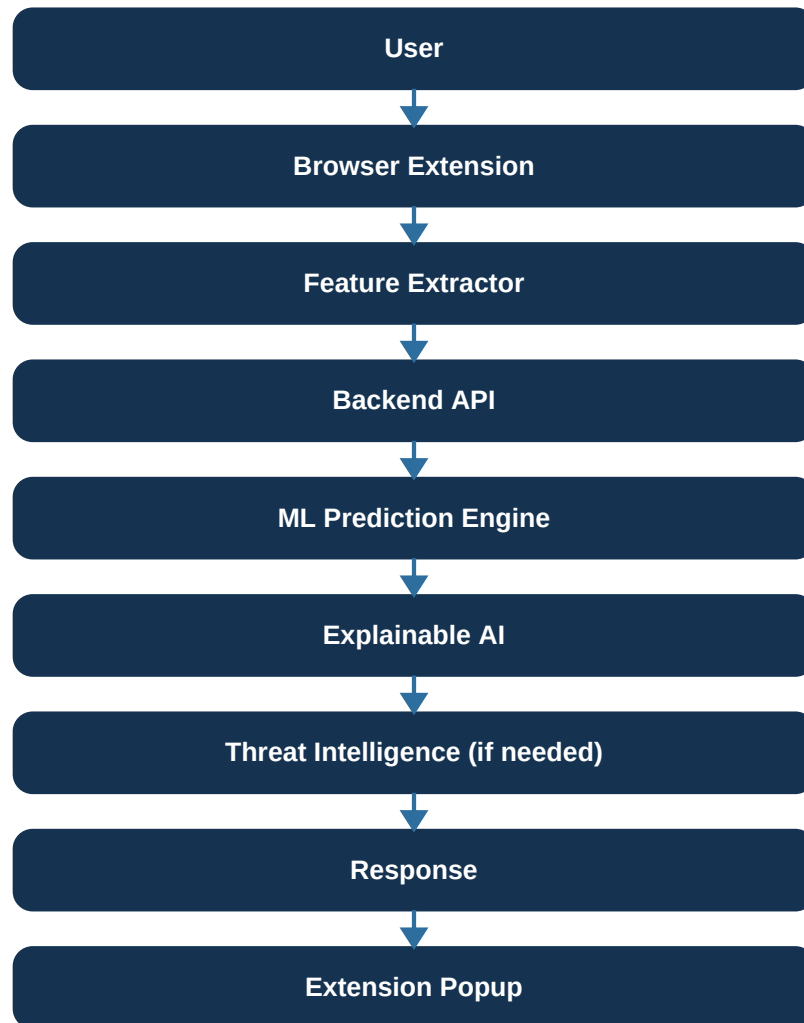
Week 10

Create a simple admin dashboard showing:

- Total websites scanned
- Phishing detected
- Safe websites
- False positives
- Most targeted brands
- Top suspicious domains
- User feedback statistics

Note: This is valuable during demonstrations.

Final Architecture



Recommended Tech Stack

Component	Technology
Browser Extension	JavaScript/TypeScript, Manifest V3
Backend	Node.js + Express
ML Training	Python + XGBoost
Explainability	SHAP
Database	MongoDB

Caching	Redis (optional)
Dashboard	React
Deployment	Docker + Render/Railway or a VPS

Suggested Improvement: Hybrid Detection Pipeline

Rather than using only XGBoost, consider a hybrid detection pipeline:

- **Fast local model (XGBoost):** uses URL and lightweight DOM features to return a result in under 100 ms.
- **Deep analysis (server-side):** runs only when the local model is uncertain. This stage performs WHOIS lookups, DNS checks, richer HTML analysis, and threat-intelligence queries.

This layered design is closer to commercial security products. It keeps the extension responsive, reduces server costs, and gives a stronger architectural story during the project presentation. It also leaves room to add optional features — like visual page similarity analysis — later, without redesigning the whole system.